Batch:	**B2**	**Roll No.:16010124114**

Experiment No._____

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of the Staff In-charge with date

Title: Study, Implementation and Analysis of Single Source Shortest Path

Objective: To learn the Greedy strategy of solving the problems for different types of problems

CO to be achieved:

CO 2 Describe various algorithm design strategies to solve different problems and analyse Complexity.

Books/ Journals/ Websites referred:

1. **1. Ellis horowitz, Sarataj Sahni, S.Rajsekaran," Fundamentals of computer algorithm", University Press**
2. **T.H.Cormen ,C.E.Leiserson,R.L.Rivest and C.Stein," Introduction to algorithms",2nd Edition ,MIT press/McGraw Hill,2001**
3. **https://www.mpi-inf.mpg.de/~mehlhorn/ftp/ShortestPathSeparator.pdf**
4. **en.wikipedia.org/wiki/Shortest_path_problem**
5. **www.cs.princeton.edu/~rs/AlgsDS07/15ShortestPaths.pdf**

Pre Lab/ Prior Concepts:

Data structures, Concepts of algorithm analysis

Historical Profile:

Sometimes the problems have more than one solution. With the size of the problem, every time it's not feasible to solve all the alternative solutions and choose a better one. The greedy algorithms aim at choosing a greedy strategy as a solutioning method and proves how the greedy solution is better one.

Though greedy algorithms do not guarantee optimal solutions, they generally give a better and feasible solution.

The path finding algorithms work on graphs as input and represent various problems in the real world.

New Concepts to be learned: Application of algorithmic design strategy to any problem, Greedy method of problem solving Vs other methods of problem solving, optimality of the solution

Topic: GREEDY METHOD

Theory: The greedy method suggests that one can devise an algorithm that work in stages, considering one input at a time. At each stage, a decision is made regarding whether a particular input is in an optimal solution. This is done by considering the inputs in an order determined by some selection procedure. If the inclusion of the next input into the partially constructed optimal solution will result in an infeasible solution, then this input is not added to the partial solution. Otherwise, it is added. The selection procedure itself is based on some optimization measures may be plausible for a given problem. Most of these, however, will result in algorithms that generate suboptimal solutions. This version of the greedy technique is called the **subset paradigm**.

Control Abstraction:

```
SolType Greedy (Type s [], int n)

// a[1:n] contains the n inputs.

{SolType solution = EMPTY;
        // Initialize the solution.
        For (int i=1; I<=n; i++) {
                Type x = Select (a);

        If Feasible (solution, x)

        Solution = Union (solution, x) ;

}
return solution;
}
```

Algorithm:

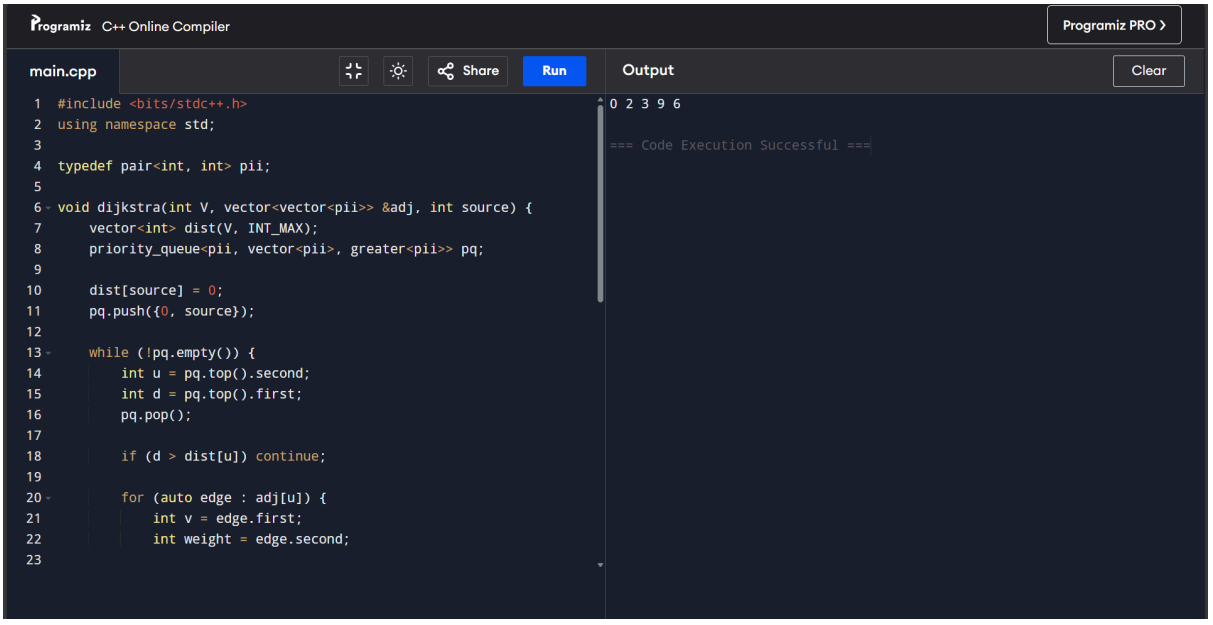
```
Algorithm ShortestPaths(v, cost, dist, n)
// dist[j], 1 ≤ j ≤ n, is set to the length of the shortest
// path from vertex v to vertex j in a digraph G with n
// vertices. dist[v] is set to zero. G is represented by its
// cost adjacency matrix cost[1 : n, 1 : n].
{
    for i := 1 to n do
    { // Initialize S.
        S[i] := false; dist[i] := cost[v, i];
    }
    S[v] := true; dist[v] := 0.0; // Put v in S.
    for num := 2 to n − 1 do
    {
        // Determine n − 1 paths from v.
        Choose u from among those vertices not
        in S such that dist[u] is minimum;
        S[u] := true; // Put u in S.
        for (each w adjacent to u with S[w] = false) do
            // Update distances.
            if (dist[w] > dist[u] + cost[u, w])) then
                    dist[w] := dist[u] + cost[u, w];
    }
}
```

Code:-

```
#include <bits/stdc++.h>

using namespace std;


typedef pair<int, int> pii;


void dijkstra(int V, vector<vector<pii>> &adj, int source) {

    vector<int> dist(V, INT_MAX);

    priority_queue<pii, vector<pii>, greater<pii>> pq;


    dist[source] = 0;

    pq.push({0, source});
```

```
    while (!pq.empty()) {
        int u = pq.top().second;
        int d = pq.top().first;
        pq.pop();

        if (d > dist[u]) continue;

        for (auto edge : adj[u]) {
            int v = edge.first;
            int weight = edge.second;

            if (dist[v] > dist[u] + weight) {
                dist[v] = dist[u] + weight;
                pq.push({dist[v], v});
            }
        }
    }

    for (int i = 0; i < V; i++) {
        cout << dist[i] << " ";
    }
}

int main() {
    int V = 5;
    vector<vector<pii>> adj(V);
```

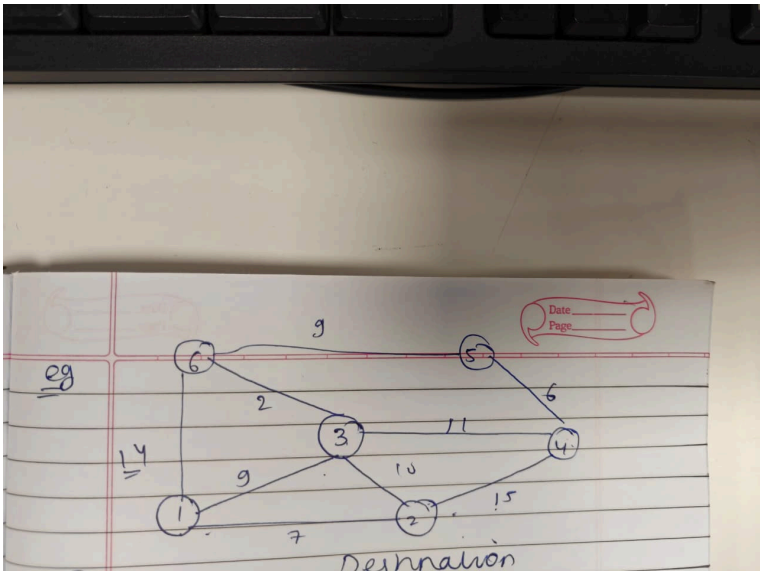
```
    adj[0].push_back({1, 2});
    adj[0].push_back({2, 4});
    adj[1].push_back({2, 1});
    adj[1].push_back({3, 7});
    adj[2].push_back({4, 3});
    adj[3].push_back({4, 1});

    int source = 0;

    dijkstra(V, adj, source);

    return 0;
}
```



```
    }

    for (int i = 0; i < V; i++) {
        cout << dist[i] << " ";
    }
}

int main() {
    int V = 5;
    vector<vector<pii>> adj(V);

    adj[0].push_back({1, 2});
    adj[0].push_back({2, 4});
    adj[1].push_back({2, 1});
    adj[1].push_back({3, 7});
    adj[2].push_back({4, 3});
    adj[3].push_back({4, 1});

    int source = 0;

    dijkstra(V, adj, source);

    return 0;
```

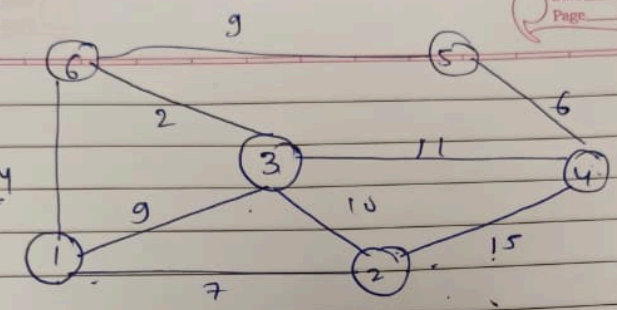
```
0 2 3 9 6

=== Code Execution Successful ===
```

Example Graph:

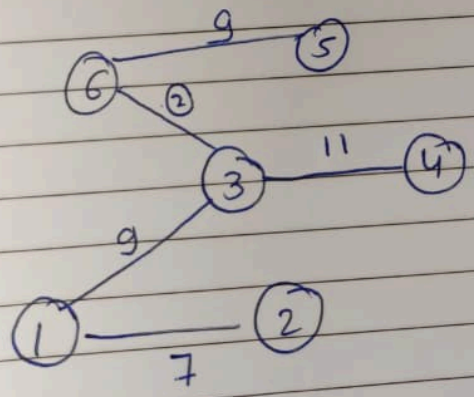


Solution:

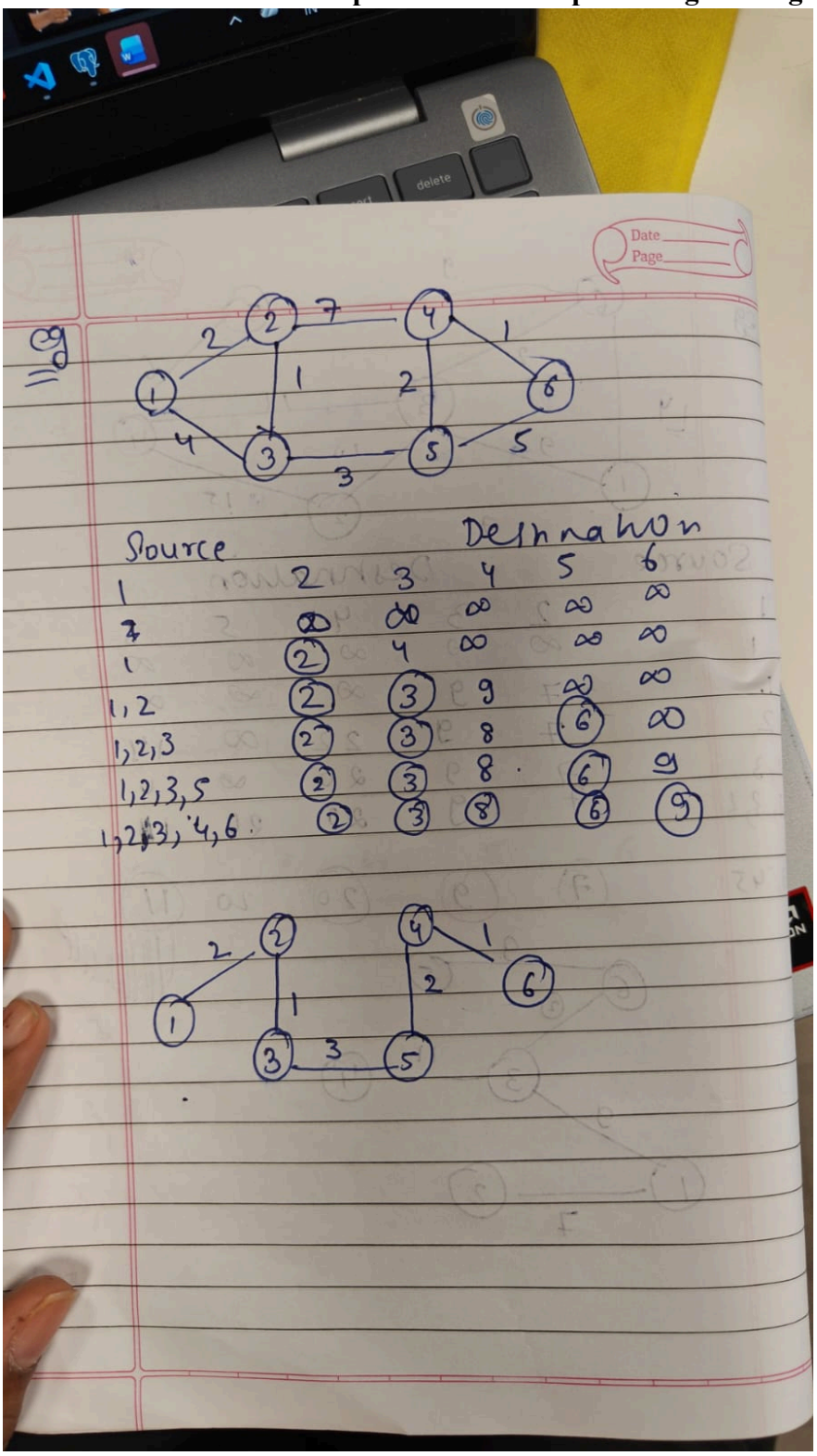
eg



Source	Destination				
	2	3	4	5	6
1	∞	∞	∞	∞	∞
1	7	9	∞	∞	∞
1.	7	9	22	∞	14
1,2	7	9	20	∞	14
1,2,3	7	9	20	20	14
1,2,3,6	7	9	20	20	11
1,2,3,45	7	9	20	20	11



Example:-
Solution:-



Code:-

Time Complexity for single source shortest path
:-O(n²)

Conclusion:
Successfully understand and trace the Dijkstras Algorithm.